

Concordia University

Lab 3: Functions in MATLAB and the Sampling Theorem

ELEC 342
Lab Section: UL-X

Discrete Time Signals and Systems

Penoelo Thibaud
40212017

Lab Instructor: Razieh Abdolahi

Performed on : October 22, 2024
Submitted on : November 5th, 2024

I certify that this submission is my original work and meets the Faculty's Expectations of Originality

Objectives –

The purpose of this lab was to explore the use of MATLAB for creating user-defined functions, investigating the Sampling Theorem, and analyzing signals in both time and frequency domains. The lab was divided into three main sections:

Part I: Focused on implementing and understanding MATLAB functions, with exercises demonstrating key programming techniques such as local and global variables, pass-by-value, and pass-by-reference mechanisms. These exercises aimed to enhance familiarity with MATLAB's function-handling capabilities.

Part II: Aimed at examining the effect of sampling rate on signal representation, utilizing the Sampling Theorem to demonstrate the relationship between sampling rates and the Fourier transform of signals. This section highlighted the significance of the Nyquist rate in preventing aliasing and preserving signal integrity.

Theory -

Part 1

Functions in MATLAB

A block of code designed to perform a specific task and can be reused with different inputs. Functions can be saved in separate M-files.

Table 1 Example of Call-by-Value

```
function inc_x = inc(x)
    x = x + 1;
    inc_x = x;
end
```

Arguments are passed by value, meaning that a copy of the input is passed to the function.

Table 2 Example of Call-by-Reference

```
function result = squareArray(arr)
    result = arr.^2;
end
```

It is the same as call-by-value but optimizes memory usage with call-by-reference for large arrays, as long as the array isn't modified within the function.

Table 3 Persistent Variables

```
function count = counter()
    persistent countVar;
    if isempty(countVar)
        countVar = 0;
    end
    countVar = countVar + 1;
    count = countVar;
end
```

Persistent variables retain their values between function calls, similar to static variables in other programming languages.

Table 4 Global Variables

```
global sharedVar
sharedVar = 10;
incrementGlobal();
disp(sharedVar);

function incrementGlobal()
    global sharedVar;
    sharedVar = sharedVar + 1;
end
```

Global variables are accessible across multiple functions and the base workspace.

Part 2

Sampling Theorem

Sampling is the process of converting a continuous signal (e.g., a sine wave) into a discrete signal by measuring its amplitude at regular intervals. The number of samples taken per period of the signal affects how accurately the signal is represented in digital form.

```
n = [0 : 7];
N = 4;
x = sin((2*pi)/N * n);
stem(n, x);
title('Two periods of sine wave with 4 samples per period');
xlabel('n');
ylabel('x[n]');
```

With 4 samples per period, the sine wave appears as discrete points. This low sample rate captures the basic shape but lacks finer details. Increasing the sample rate will improve the waveform's resemblance to the original continuous sine wave.

```
n = [0 : 7];
N = 8;
x = sin((2*pi)/N * n);
stem(n, x);
```

```
title('Two periods of sine wave with 4 samples per period');  
xlabel('n');  
ylabel('x[n]');
```

Increasing the sample rate to 8 samples per period makes the sampled signal more closely resemble the continuous sine wave.

Nyquist Rate

It is the minimum sampling rate required to avoid aliasing, which is twice the highest frequency component of the signal. If a signal is under sampled, different frequency components can become indistinguishable, leading to distortion in the sampled signal.

Fourier Transform of Sampled Signals

It decomposes a time-domain signal into its frequency components, allowing us to see the frequency content of a signal. Peaks in the Fourier Transform plot correspond to the frequencies present in the signal.

Tasks/Results/Discussion

Questions 1

Answer (1A):

```
function main_part_a()
    global w;

    % Get user inputs
    periods = input('Enter the number of periods: ');
    step_size = input('Enter the step size of the frequency interval: ');
    w = [-periods * pi : step_size : periods * pi];

    for i = 1:5
        % Get sampling rate in terms of Nyquist rate
        sampling_rate = input('Enter the sampling rate (multiple of Nyquist rate): ');

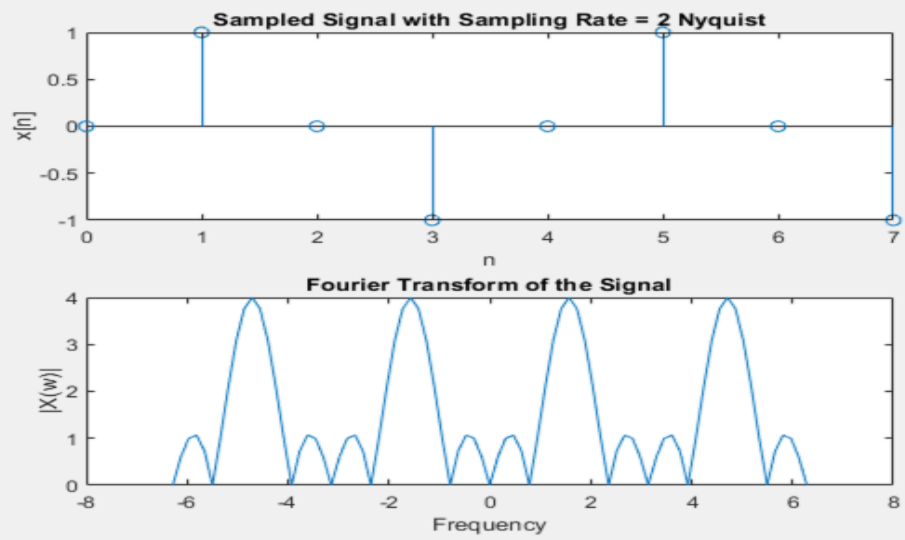
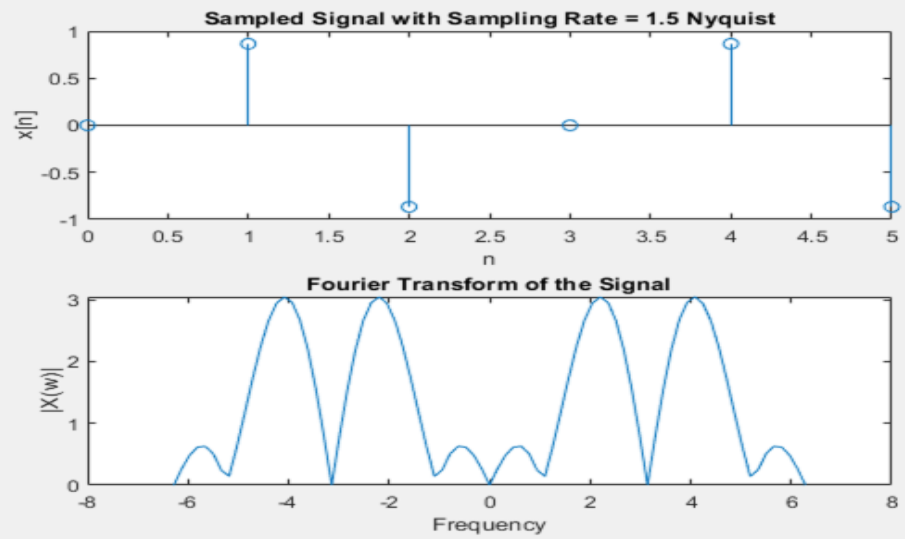
        % Compute N and the total number of samples n
        N = floor(sampling_rate * 2);
        n = 2 * N;

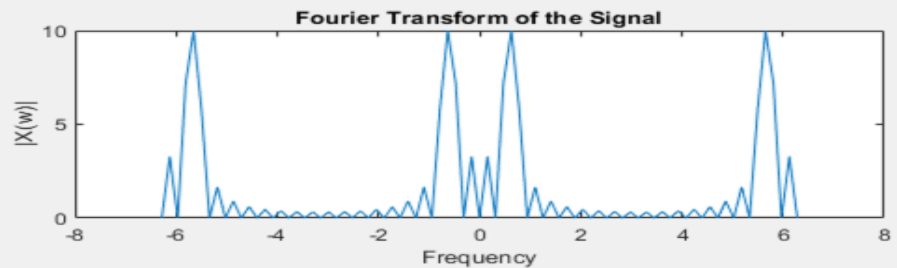
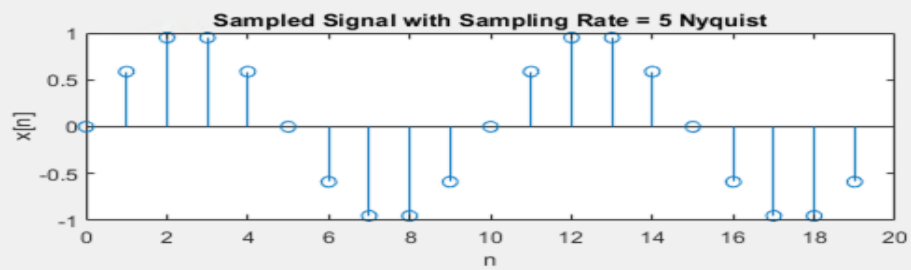
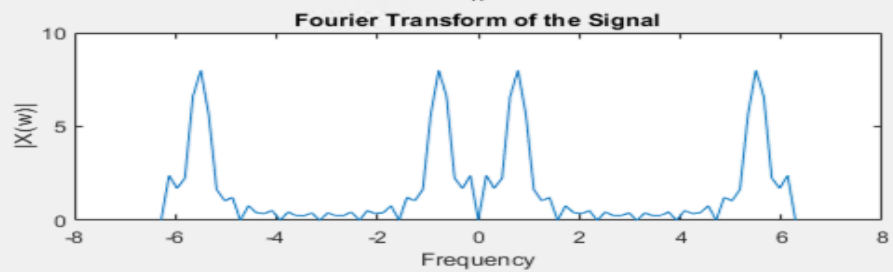
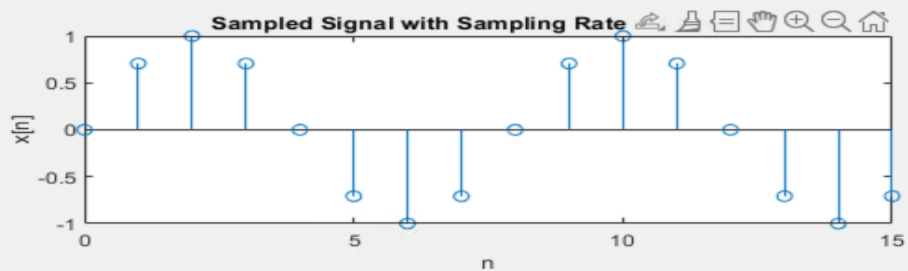
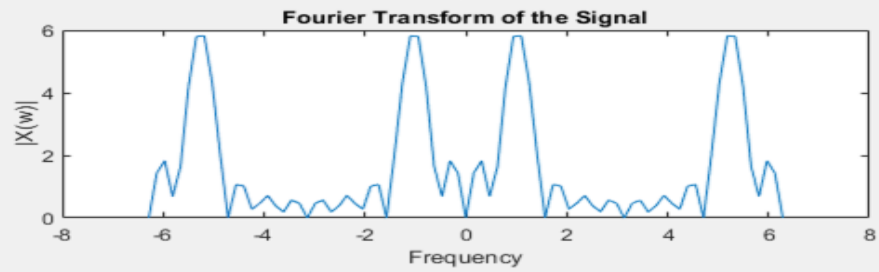
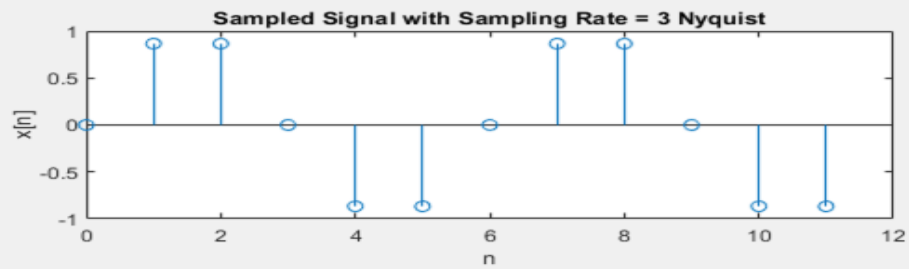
        % Generate the sampled signal
        x = sin(2 * pi / N * (0:n-1));

        % Plot the sampled signal
        figure;
        subplot(2, 1, 1);
        stem(0:n-1, x);
        title(['Sampled Signal with Sampling Rate = ' num2str(sampling_rate) ' Nyquist']);
        xlabel('n');
        ylabel('x[n]');

        % Compute and plot the Fourier Transform
        X_w = dtfft(x);
        subplot(2, 1, 2);
        plot(w, abs(X_w));
        title('Fourier Transform of the Signal');
        xlabel('Frequency');
        ylabel('|X(w)|');
    end
```

end





The Nyquist value of 1.5 is slightly above the Nyquist rate to introduce slight aliasing effects. The Nyquist value of 2 is Exactly the Nyquist rate, it should be the minimum rate to avoid aliasing. The Nyquist value of 3, 4 and 5 are higher than the Nyquist rate, which gives better signal reconstruction and reduces aliasing. The DTFT code is available in the annexes section.

Answer (1B):

```
function main_part_b()
    global w;

    % Ask the user for a fixed sampling rate (in terms of Nyquist rate)
    sampling_rate = input('Enter the fixed sampling rate (multiple of Nyquist
rate): ');
    N = floor(sampling_rate * 2); % Samples in one period

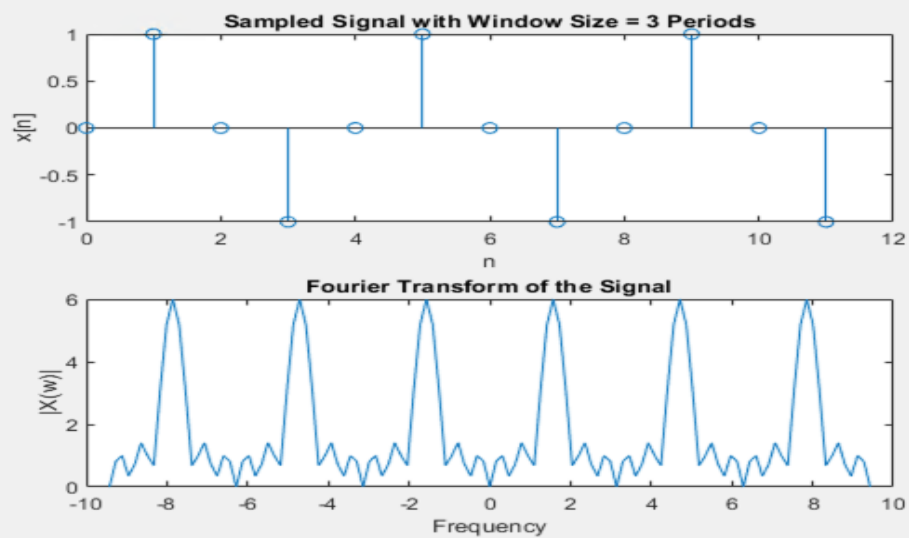
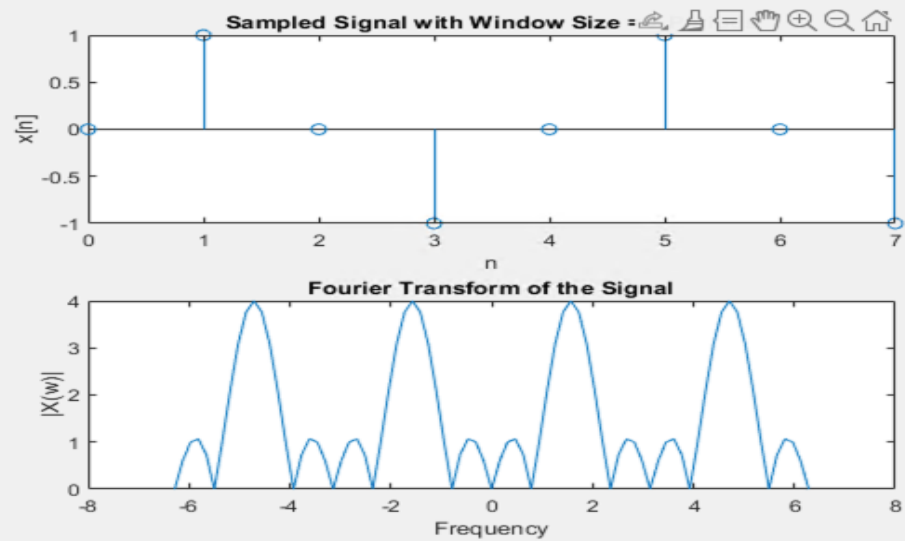
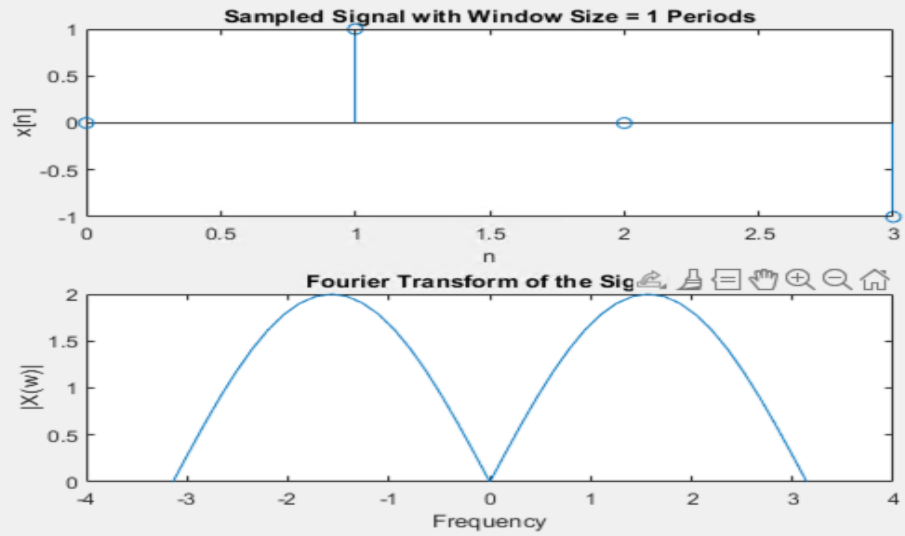
    for i = 1:5
        % Get the window size in terms of number of periods
        window_size = input('Enter the window size (in terms of periods of the
signal): ');
        n = window_size * N; % Total number of samples

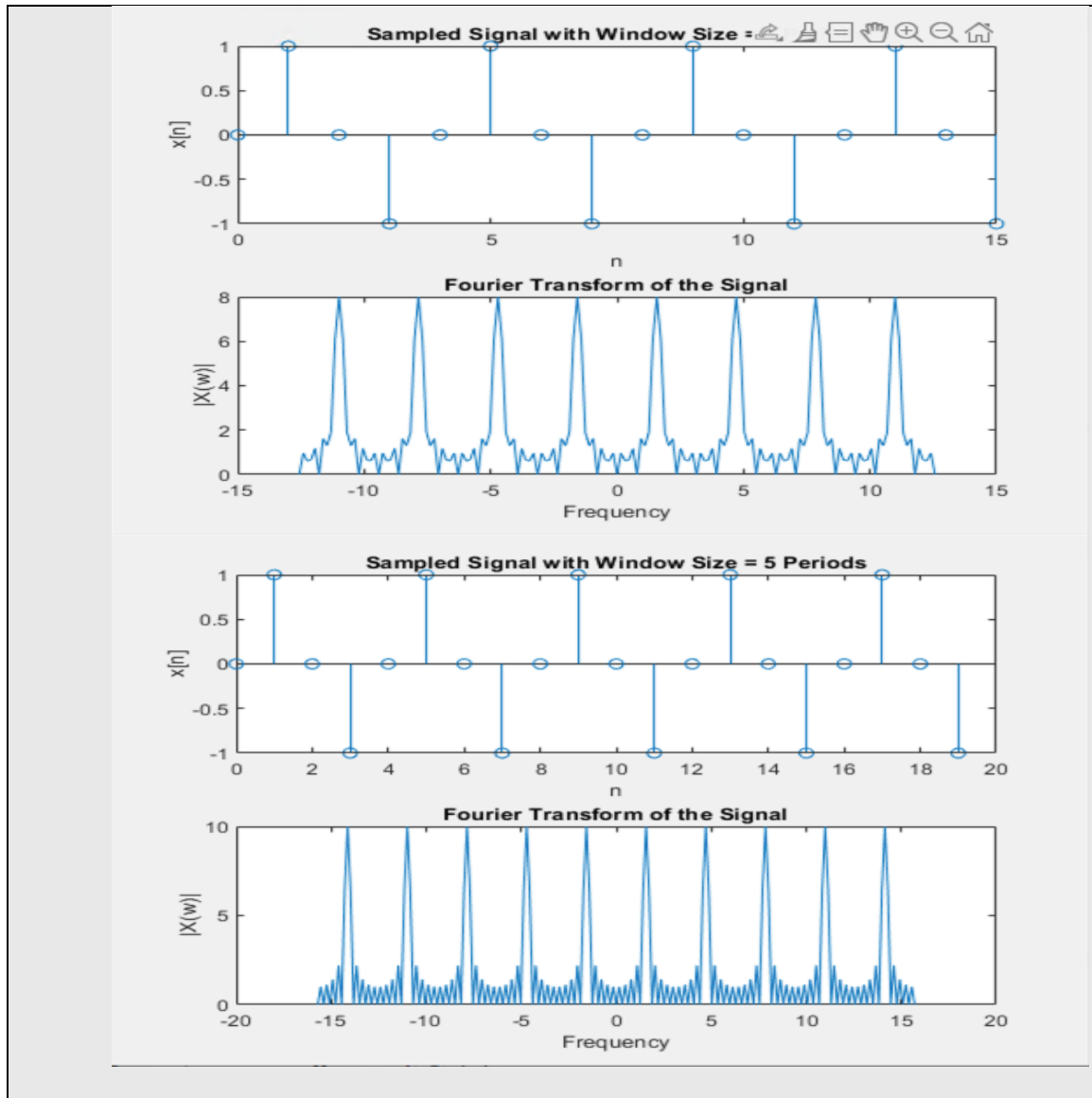
        % Update the frequency interval based on window size
        w = [-window_size * pi : 0.05 * pi : window_size * pi];

        % Generate the signal
        x = sin(2 * pi / N * (0:n-1));

        % Plot the signal and Fourier Transform
        figure;
        subplot(2, 1, 1);
        stem(0:n-1, x);
        title(['Sampled Signal with Window Size = ' num2str(window_size) '
Periods']);
        xlabel('n');
        ylabel('x[n]');

        % Compute and plot the Fourier Transform
        X_w = dtfft(x);
        subplot(2, 1, 2);
        plot(w, abs(X_w));
        title('Fourier Transform of the Signal');
        xlabel('Frequency');
        ylabel('|X(w)|');
    end
end
```





We can notice that increasing the window size makes the Fourier Transform more defined and improves frequency resolution. The sampled signal will become more representative of the original signal as you increase the number of periods. The Fourier Transform will show clearer peaks and a smoother spectrum with larger window sizes. The DTFT code is available in the annexes section.

Questions 2:

```
function main_part2()
    global w;

    % Get user input for the number of periods and step size
    periods = input('Enter the number of periods: ');
    step_size = input('Enter the step size of the frequency interval: ');
    w = [-periods * pi : step_size : periods * pi];

    for i = 1:5
        % Get sampling rate in terms of Nyquist rate
        sampling_rate = input('Enter the sampling rate (multiple of Nyquist
rate): ');

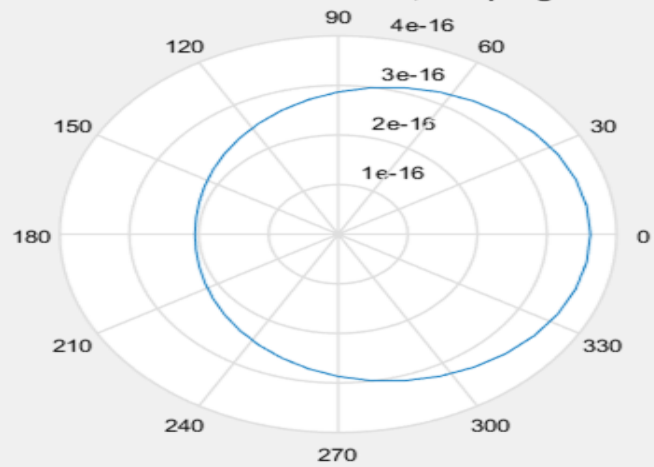
        % Compute N and the total number of samples n
        N = floor(sampling_rate * 2);
        n = 2 * N;

        % Generate the specified signal
        x = 0.5 * sin((2 * pi / N) * (0:n-1)) + 0.33 * sin((4 * pi / N) * (0:n-1));

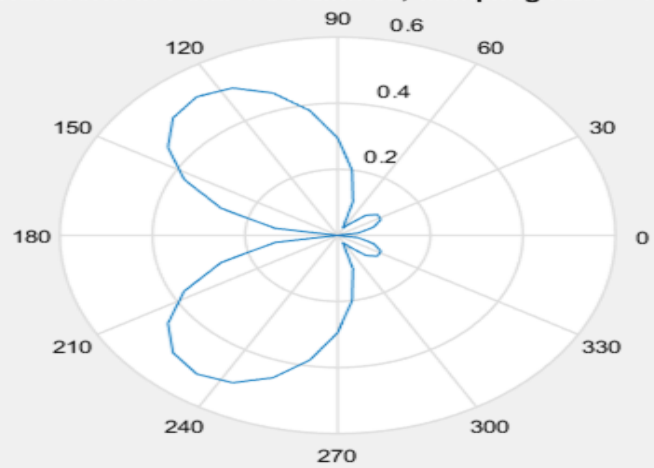
        % Compute the Fourier Transform
        X_w = dtft(x);

        % Plot the Fourier Transform in polar coordinates
        figure;
        polar(w, abs(X_w));
        title(['Fourier Transform in Polar Coordinates, Sampling Rate = '
num2str(sampling_rate) ' Nyquist']);
    end
end
```

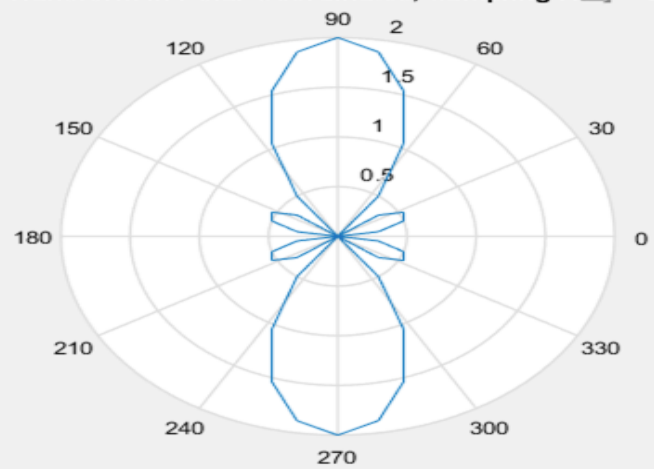
Fourier Transform in Polar Coordinates, Sampling Rate = 1 Nyquist

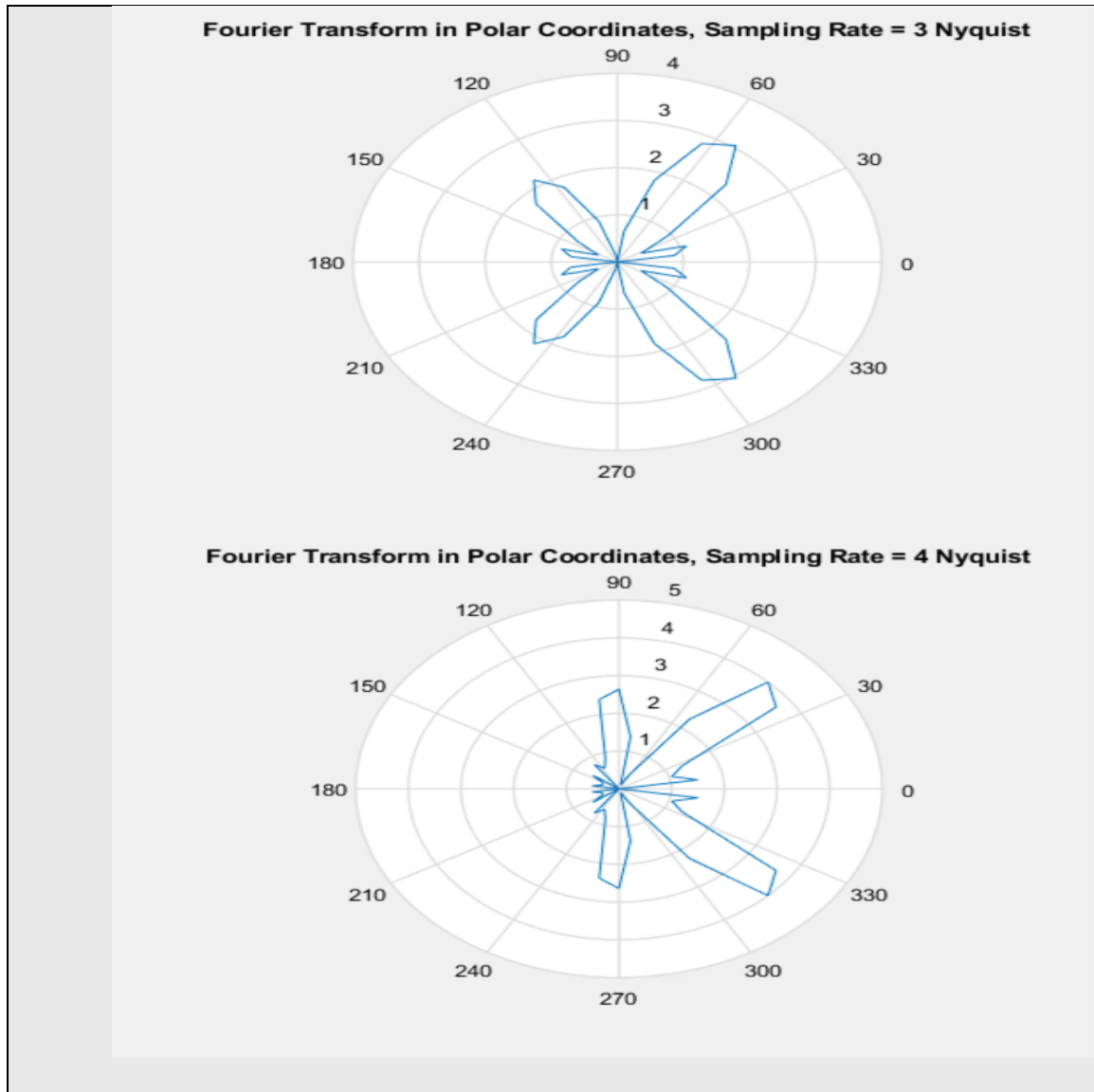


Fourier Transform in Polar Coordinates, Sampling Rate = 1.5 Nyquist



Fourier Transform in Polar Coordinates, Sampling Rate = 2 Nyquist





The Following results will provide insights into the frequency components and their corresponding magnitudes and phases. We can notice that the number of lobes and their orientation varied as the sampling rate was increasing.

Questions 3:

Answer (3A):

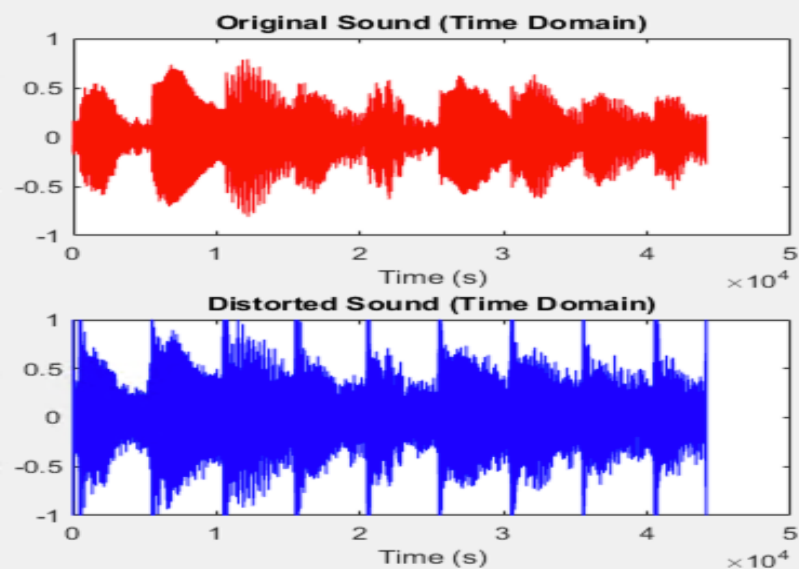
```
[x_original, fs] = audioread('wav/Original.wav');  
[x_distorted, fs] = audioread('wav/Distorted.wav');
```

```
x_original=x_original(:,1);  
x_distorted=x_distorted(:,1);
```

```
L=2*fs;  
t=[0:L-1];
```

```
figure;  
subplot(3,2,1)  
plot(t,x_original,'color','r')  
title('Original Sound (Time Domain)')  
xlabel('Time (s)')
```

```
subplot(3,2,3)  
plot(t,x_distorted,'color','b')  
title('Distorted Sound (Time Domain)')  
xlabel('Time (s)')
```



Answer (3B):

```
difference=x_original-x_distorted;
squared=difference.^2;
sum_squares=sum(squared);
MSE=(1/L)*sum_squares;

disp('MSE between Original and Distorted Sound is ')
disp(MSE)
```

```
MSE between Original and Distorted Sound is
0.0250
```

Answer (3C):

```
% Load the original and distorted signals
[x_original, Fs] = audioread('wav/Original.wav');
[x_distorted, ~] = audioread('wav/Distorted.wav');

% Ensure the signals are column vectors
x_original = x_original(:);
x_distorted = x_distorted(:);

% FFT of the original and distorted signals
X_original = fft(x_original);
X_distorted = fft(x_distorted);

% Initialize the recovered signal in frequency domain
X_recovered = zeros(size(X_distorted));

% Define the cutoff frequencies in radians/sample
L = length(X_distorted);
for i = 1:L
    frequency = 2 * pi * (i - 1) / L;
```

```

    % Retain frequencies below  $\pi/2$  and above  $3\pi/2$ 
    if frequency <  $\pi/2$  || frequency >  $3\pi/2$ 
        X_recovered(i) = X_distorted(i);
    end
end

% Inverse FFT to recover the time-domain signal
x_recovered = real(ifft(X_recovered));
x_recovered = x_recovered(:);

% Save the recovered signal to a new WAV file
audiowrite('Recovered.wav', x_recovered, Fs);

% Time vector for plotting
t = (0:L-1) / Fs;

% Frequency vector for plotting (normalized frequency in radians)
w = linspace(0, 2 * pi, L);

% Plotting the original, distorted, and recovered signals in time and
frequency domains
figure;

% Time domain plots
subplot(3, 2, 1);
plot(t, x_original, 'r');
title('Original Signal in Time Domain');
xlabel('Time (s)');
ylabel('Amplitude');

subplot(3, 2, 3);
plot(t, x_distorted, 'b');
title('Distorted Signal in Time Domain');
xlabel('Time (s)');
ylabel('Amplitude');

subplot(3, 2, 5);
plot(t, x_recovered, 'm');
title('Recovered Signal in Time Domain');
xlabel('Time (s)');
ylabel('Amplitude');

% Frequency domain plots
subplot(3, 2, 2);

```

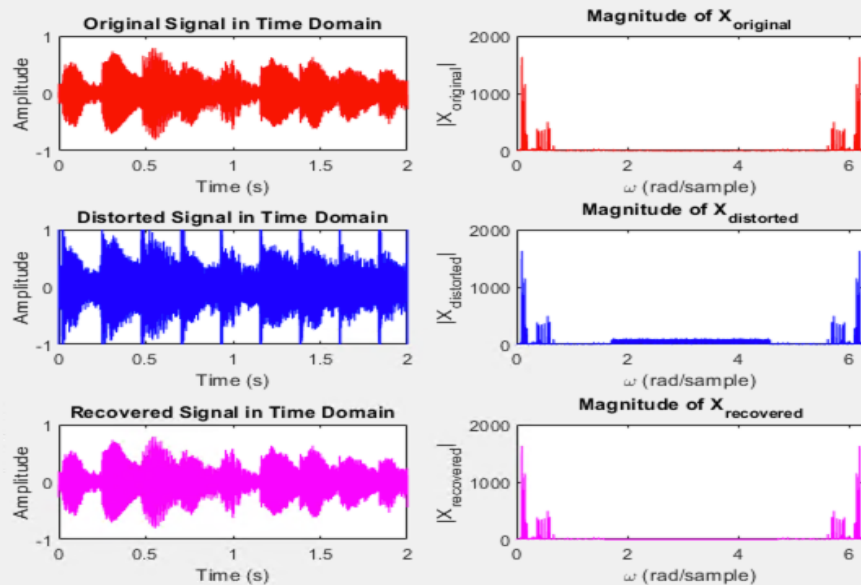
```

plot(w, abs(X_original), 'r');
title('Magnitude of X_{original}');
xlabel('\omega (rad/sample)');
ylabel('|X_{original}|');

subplot(3, 2, 4);
plot(w, abs(X_distorted), 'b');
title('Magnitude of X_{distorted}');
xlabel('\omega (rad/sample)');
ylabel('|X_{distorted}|');

subplot(3, 2, 6);
plot(w, abs(X_recovered), 'm');
title('Magnitude of X_{recovered}');
xlabel('\omega (rad/sample)');
ylabel('|X_{recovered}|');

```



Answer (3D):

```
difference=x_original-x_recovered;  
squared=difference.^2;  
sum_squares=sum(squared);  
MSE=(1/L)*sum_squares;  
  
disp('MSE between Original and Recovered Sound is ')  
disp(MSE)
```

```
MSE between Original and Recovered Sound is  
3.0564e-04
```

Answer (3E):

Listening to the original, the distorted and the recovered version of the audio made it obvious the original sound is efficiently recovered and that most of the noise was removed. Creating a frequency domain helped removing the noise and the Fourier transform help accentuate the noisy part of the signal to extract.

Conclusions –

This lab reinforced MATLAB's utility in discrete-time signal processing, from theoretical explorations of sampling and aliasing to practical applications in noise reduction. The successful recovery of the original audio signal demonstrated the efficacy of frequency-domain filtering for signal restoration, aligning with core principles of digital signal processing.

Annexes –

Codes –

DTFT.m

```
% Discrete Time Fourier Transform Function
function X_w = dtft(x)
    global w;
    X_w = zeros(size(w));
    for k = 1:length(w)
        X_w(k) = sum(x .* exp(-1i * w(k) * (0:length(x)-1)));
    end
end
```